

An Experimental Comparison Between an Arduino Due and a DSP-based Delfino LaunchPad board

Marcelo Reyes
Faculty of Engineering.
Universidad del Bío-Bío.
Concepción, Chile.
mareyes@ieec.org

Pedro Melín
Dept. of Electrical and Electronic
Engineering.
Universidad del Bío-Bío.
Concepción, Chile.
pemelin@ubiobio.cl

Eduardo Espinosa
Dept. of Electrical Engineering.
Universidad Católica de la
Santísima Concepción.
Concepción, Chile.
eespinosa@ucsc.cl

Marco Rivera
Faculty of Engineering.
Universidad de Talca.
Curicó, Chile.
marcoriv@utalca.cl

José Suárez
Faculty of Engineering.
Universidad de Talca.
Curicó, Chile
jose.suarez@utalca.cl

Abstract— In the present work, the performance of the Arduino Due and Delfino LaunchPad TMS320F28379D development boards are evaluated and compared, to be a guide for developers who must decide to migrate their application, or who must decide between both technologies at the beginning of their project. When considering the compatibility of their electrical levels for the connection of peripherals and differences in computing capacity. In particular, the execution time of mathematical operations and instructions of common use in the acquisition and processing of signals is evaluated using available functions in C language. The work considers (i) the evaluation of the characteristics of hardware and development environments, (ii) the methodology for the evaluation of execution times, algorithms, and configuration considerations, (iii) case study and results, and (iv) performance comparison. The results show that basic mathematical operations with floating-point are executed 16 times faster when migrating from Arduino Due to Delfino LaunchPad.

Keywords— *arduino, delfino, development board, launchPad, TMS320F28379D*

I. INTRODUCCIÓN

El desarrollo de tecnologías con requerimientos de rápidas respuestas a solicitudes tales como el envío de datos [1], la ejecución de algoritmos para el procesamiento de datos [2], la adquisición de señales de audio [3], el procesamiento de señales eléctricas [4][5] y señales biomédicas [6][7], requieren del uso de dispositivos digitales electrónicos [8][9]. Típicamente los dispositivos utilizados pueden ser clasificados en (i) microprocesadores, (ii) los microcontroladores, (iii) los dispositivos del tipo *Digital Signal Processor* (DSP) y (iv) del tipo *Field-Programmable Gate Array* (FPGA) [8]-[11], entre otros. Cada dispositivo tiene asociados entornos de desarrollo (*Integrated Development Environment*, IDE) para su configuración o el desarrollo de soluciones mediante funciones genéricas o de fácil acceso.

Las tarjetas de desarrollo permiten implementar soluciones electrónicas de forma rápida, al integrar en una única tarjeta la unidad de procesamiento, el sistema de energía y algunos periféricos compatibles con la unidad de procesamiento. Esto permite a un desarrollador conectar dispositivos externos de forma simple y centrarse en la configuración y algoritmos requeridos por la aplicación que desarrolla. De esta forma, las capacidades de las tarjetas de desarrollo se encuentran limitadas por la unidad de procesamiento seleccionada, el número de periféricos utilizados y pines disponibles al usuario, tales como número de entradas y salidas análogas o digitales, puertos de comunicación, almacenamiento, entre otras [12][13].

Entre las alternativas que existen en el mercado, destacan dos tarjetas de desarrollo que por su precio son accesibles para estudiantes, ingenieros y desarrolladores en general. Por un lado, la placa de desarrollo Arduino Due, que utiliza el microcontrolador Atmel ATSAM3X8E, es una plataforma de hardware libre que por su popularidad es atractiva para iniciar en el mundo de la programación y desarrollo electrónico [14]-[16] e implementar proyectos en áreas como la domótica [17], robótica, electrónica de potencia [18][19] y accionamientos eléctricos [20], debido a que cuenta con una amplia disponibilidad de librerías, tarjetas de expansión, ejemplos y documentación. Por otro lado, la tarjeta de desarrollo LAUNCHXL-F28379D C2000 Delfino LaunchPad (desde ahora Delfino LaunchPad) es una plataforma de Texas Instrument que incorpora el microcontrolador TMS320F28379D, el que es un microcontrolador diseñado para el control en tiempo real y que es ampliamente utilizado en el control de convertidores estáticos de potencia, el control de motores, el procesamiento digital de señales, entre otros [21]-[26]. En cuanto a la programación, ambas cuentan con acceso a IDE pertenecientes a cada fabricante, cuentan con la opción de utilizar lenguaje C y disponen de librerías que ayudan a disminuir el tiempo en la curva de aprendizaje. Para Arduino Due se encuentra IDE-Arduino, software de código abierto caracterizado por ser un entorno multiplataforma. Para la plataforma Delfino LaunchPad se encuentra Code Composer

Este trabajo ha sido financiado por la Agencia Nacional de Investigación y Desarrollo ANID, bajo el proyecto ANID/FONDECYT/1201683, y la MacroFacultad de Ingeniería 2030.

Studio (CCS), entorno profesional creado para procesadores integrados y microcontroladores de Texas Instrument.

En comparación con Arduino Due, Delfino LaunchPad tiene un costo similar en el mercado (40 USD) y ambas operan en niveles eléctricos CMOS 3,3V. Utiliza un microcontrolador que cuenta con unidades de cálculo especializadas. Por otro lado, no cuenta con una comunidad que desarrolle librerías y aplicaciones como la de Arduino, y utiliza una IDE de mayor complejidad. Para un desarrollador que utiliza Arduino Due, es posible que por la necesidad de mayores requerimientos de procesamiento de su aplicación requiera migrar la plataforma de desarrollo, instante en que la tarjeta Delfino LaunchPad se convierte en una alternativa debido a sus mayores capacidades de cómputo. Sin embargo, migrar de plataforma, desde Arduino Due a Delfino LaunchPad, requiere evaluar las variaciones de hardware, complejidades de programación en los entornos de desarrollo, como también la posible mejora en las capacidades de procesamiento de las tarjetas de desarrollo.

En el presente trabajo se propone la evaluación y comparación del desempeño de las tarjetas de desarrollo Arduino Due y Delfino LaunchPad, con el objetivo de ser una guía para desarrolladores que deban tomar la decisión de migrar su aplicación, o que deban decidir entre ambas tecnologías al inicio de su proyecto. En particular, se evalúa el tiempo de ejecución de operaciones matemáticas e instrucciones de uso común en la adquisición y procesamiento de señales utilizando funciones predeterminadas o disponibles en lenguaje C. Este documento se estructura de la siguiente manera: en el capítulo II se estudian ambas plataformas de desarrollo, donde se dan a conocer y evalúan sus características y sus entornos de desarrollo; el capítulo III describe la metodología donde se presenta el método de cálculo de los tiempos de ejecución, algoritmos para evaluar operaciones e instrucciones, y consideraciones de configuración. El capítulo IV presenta el caso de análisis y resultados obtenidos. En el capítulo V se realiza la comparación de desempeño, y finalmente, el capítulo VI contiene las conclusiones de este trabajo. Entre los resultados, podemos mencionar que la migración desde Arduino Due a Delfino LaunchPad acelera los tiempos de ejecución de operaciones básicas de punto flotante a lo menos en 16 veces,

manteniendo la compatibilidad eléctrica entre la aplicación y la tarjeta.

II. PLATAFORMAS DE DESARROLLO

Las plataformas de desarrollo a evaluar consisten en Arduino Due, hardware libre que utiliza el microcontrolador de 32 bit CortexM3 ARM de Atmel ATSAM3X8E, y la placa de desarrollo de Texas Instruments LAUNCHXL-F28379D C2000 Delfino LaunchPad, la cual utiliza el microcontrolador de control en tiempo real TMS320F28379D. A continuación, se revisarán (i) las características de las tarjetas de desarrollo y (ii) sus entornos de desarrollo IDE, con el objetivo de evaluar los recursos disponibles y complejidades de configuración.

A. Características de las tarjetas de desarrollo

Arduino Due opera con una CPU de 32 bits y frecuencia de 84 MHz. Cuenta con memoria RAM de 512 KB y 96 KB de SRAM, además de salidas y entradas digitales, ADC, DAC y periféricos de comunicación, entre otros, Tabla 1. De los periféricos, Arduino Due dispone al usuario 54 entradas y salidas digitales de los 103 disponibles en el microcontrolador, como también 12 de los 16 canales PWM, 12 de las 16 entradas análogas de 12 bits, 1 de los 2 puertos CAN, entre otros, Tabla 2. Utilizar una mayor cantidad de periféricos o modificar los pines de acceso requerirá una configuración avanzada al no contar con soporte oficial en el IDE de Arduino. Sin embargo, es posible utilizar librerías o programas desarrollados por la comunidad de usuarios de las plataformas Arduino para lograr lo anterior.

Por otro lado, la tarjeta Delfino LaunchPad, al utilizar el microcontrolador TMS320F28379D, opera con una CPU dual y CLA dual (*Control Law Accelerator*, acelerador matemático de punto flotante de operación paralela a cada CPU) de 32 bits y frecuencia de operación hasta 200 MHz. Cada núcleo cuenta con una FPU (unidad de procesamiento de punto flotante de 32 bits, parte de la CPU), una TMU (unidad matemática trigonométrica) y una VCU (Viterbi, Complex math and CRC unit) que acelera las operaciones matemáticas complejas, como la transformada rápida de Fourier. Además, cuenta con memoria Flash de 1 MB, RAM de 204 KB, conversores análogos a digital (ADC) de 16 o 12 bits, conversores digital-análogo (DAC) de 12 bits, filtros de sincronización delta-sigma, entre otros, Tabla 1. La tarjeta de

TABLA 1. CARACTERÍSTICAS GENERALES TARJETAS DE DESARROLLO.

Tarjeta de Desarrollo	Unidad de Procesamiento		I/O Digitales	Entradas Análogas	Salidas Análogas	Voltaje Alimentación	Otros
Arduino Due	Microcontrolador ATSAM3X8E (Atmel)	-84 MHz -1 CPU -32 bits -512 KB FLASH -96 KB SRAM -9 Timer de 32 bits	54 (12 salidas PWM)	12 canales (1 ADC de 12 bits)	2 DAC	7-12 V USB 5 V (I/O a 3.3 V)	CAN sin transductor. Sin aislación JTAG/USB
Delfino LaunchPad	MCU TMS320F28379D (Texas Instrument)	-200 MHz -2 CPU -32 bits -1 MB FLASH -204 KB RAM -6 Timer de 32 bits (3 por CPU) - 2 CLA - - FPU - - VCU - - TMU -	50 (12 salidas PWM)	16 canales con 3 ADC de 12 bits. 4 canales diferenciales, 1 ADC de 16 bits.	2 DAC* 4 con PWM y Filtro	3.3 V USB 5.0 V (I/O a 3.3 V)	CAN con transductor. Con posibilidad de aislación JTAG/USB

*Prevía configuración

TABLA 2. CARACTERÍSTICAS Y USO DE PROCESADOR EN TARJETAS DE DESARROLLO.

Periférico	ATSAM3X8E	Arduino DUE	TMS320 F28379D	Delfino LaunchPad
P I/O (GPIO)	103 de 144 pines	54 GPIO	169 GPIO de 337 pines	50 GPIO
Salidas PWM	16	12	24	12
Entrada Analógica	16 (12 bit)	12 (12 bit)	24 (12 bits) o 12 diferenciales (16 bits)	16 (12 bits) y 4 diferenciales (16 bits)
ADC	1 de 12 bits	1 de 12 bits	4 de 12 o 16 bits	3 a 12 bits y 1 a 16 bits
OUT Analógica	2 (12 bit)	2 (12 bits)	3 de 12 bit	2* de 12 bits y 4 con PWM y filtro
CAN	2	1	2	2* (1 con transductor)
USART/UART	3/2	3/1*	4 (SCI/UART)	4*
I2C	2	1	2	2*
SPI	1	1	3	3*

*Prevía configuración

TABLA 3. EVALUACIÓN IDE.

Criterio	IDE	
	Arduino IDE	Code Composer Studio
Compatibilidad	ALTA (Placas Arduino y de terceros)	BAJA (Placas con procesadores Texas Instrument)
Disponibilidad Librerías C	ALTA	MEDIA
Herramientas de Análisis	BAJA	ALTA
Configuración periféricos genéricos (ADC-DAC- GPIO)	BAJO	BAJO-MEDIO
Configuración periféricos avanzados (CAN-UART- I2C- otros)	MEDIO	MEDIO-ALTO

desarrollo dispone al usuario 50 de las 169 entradas y salidas de uso general (GPIO) disponibles en el procesador, 16 de las 24 entradas analógicas de 12 bits, 4 salidas analógicas generadas desde salidas PWM con filtros pasa bajo, entre otros, Tabla 2. En cuanto a periféricos, como las salidas analógicas por DAC y de comunicación como UART, I2C, CAN, SPI, se requiere configurar en el IDE los puertos I/O para habilitar su disponibilidad en terminales de la placa de desarrollo.

B. Entornos de Desarrollo (IDE)

Los IDE utilizados para cada dispositivo son Arduino IDE y Code Composer Studio (CCS). Ambos disponen de programación en lenguaje C y funciones preestablecidas. En la IDE de Arduino se destaca su alta compatibilidad con otras placas Arduino y de terceros, como también su alta disponibilidad de librerías y documentación por parte de la comunidad Arduino, y baja complejidad de configuración de periféricos como ADC, DAC y GPIO, Tabla 3. Por otro lado, la

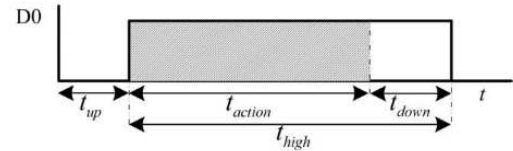


Fig. 1. Esquema método medición tiempo ejecución. Fuente: [12]

compatibilidad de CCS con otras tarjetas de desarrollo es baja al ser un IDE destinado a procesadores Texas Instrument. CSS cuenta con mayores herramientas de edición y análisis. Sin embargo, requiere de un mayor tiempo de desarrollo al incrementar el número de características a configurar y disponer de menos documentación.

En particular, al ser Arduino IDE un software Open Source, la comunidad ha desarrollado múltiples librerías y funciones para la programación de los microcontroladores. Un ejemplo de esto se presenta en permitir operar el ADC y DAC con funciones simples, sin la necesidad de una configuración compleja para el usuario. Por otro lado, el software CCS cuenta con herramientas para el desarrollo y depuración de los proyectos. CCS dispone de tres formas o herramientas para acceder a los registros del microcontrolador: (i) de manera directa, (ii) mediante campos de bits (BitFields) o (iii) mediante Driverlib. Acceder de forma directa a registros considera programar en lenguaje de bajo nivel, en cambio BitFields y en mayor medida Driverlib permite programar con lenguajes con mayor nivel de abstracción, este último mediante funciones preestablecidas para modificar registros, disponibles en la librería C2000Ware junto a documentación y ejemplos.

III. METODOLOGÍA

El análisis del rendimiento de las tarjetas de desarrollo Arduino Due y Delfino LaunchPad consiste en evaluar el tiempo de ejecución de operaciones matemáticas e instrucciones de uso común en la adquisición y procesamiento de señales, para lo cual se utilizan funciones predeterminadas disponibles en las librerías de cada IDE. En particular, para la tarjeta de desarrollo Launchpad Delfino se utilizan las funciones disponibles en Driverlib.

A. Método de evaluación del tiempo de ejecución

Los tiempos de ejecución de las operaciones o instrucciones se realizan de forma experimental utilizando las salidas digitales de las tarjetas de desarrollo como se indica en [12]. El método considera (i) la ejecución de la operación a evaluar entre los cambios de estado LOW-HIGH y HIGH-LOW de una salida digital, (ii) medición experimental del tiempo en que el estado de la salida digital es HIGH, t_{high} , y (iii) el cálculo del tiempo de ejecución de la operación evaluada, t_{action} , al restar a t_{high} el tiempo de ejecución del cambio de salida digital HIGH-LOW, t_{down} , determinado con anterioridad. Fig.1.

Lo anterior se resume en los siguientes pasos.

1) Ejecutar:

```
SalidaDigital(pin, HIGH)
FunciónEvaluada( )
SalidaDigital(pin, LOW)
```

- 2) Medición experimental t_{high} .
- 3) Calcular tiempo de ejecución de operación,

$$t_{action} = t_{high} - T_{down} \quad (1)$$

Con $T_{down} = t_{high}$ al ejecutar

```
SalidaDigital(pin, HIGH)
SalidaDigital(pin, LOW).
```

B. Algoritmos para evaluar operaciones matemáticas e instrucciones.

Las operaciones matemáticas evaluadas corresponden a la suma, sustracción, multiplicación y división considerando variables enteras y flotantes, junto a la conversión de formato entero a flotante y flotante a entero de una variable. Entre las instrucciones consideradas se encuentran la lectura de datos desde un conversor análogo digital, ADC, y salida de datos a través de un conversor digital a análogo, DAC.

El algoritmo base considera la propuesta de [12], con modificaciones en las funciones utilizadas en la tarjeta de desarrollo Delfino LaunchPad, la evaluación de la operación suma para variables constantes y actualizadas en cada ciclo de cálculo. A modo de referencia se presentan las modificaciones en los algoritmos implementados para la plataforma Delfino LaunchPad TMS320F28379D.

1) *Operaciones enteras y flotantes.* El CÓDIGO 1 presenta el algoritmo implementado para operaciones con variables de entradas enteras. Este utiliza las variables a y b para el cálculo de las operaciones. Las variables son actualizadas en cada iteración por medio de la suma de resultados anteriores. Para la operación suma también se evalúa el tiempo de ejecución cuando ambos sumandos son constantes, a0 y b0. El algoritmo para la evaluación de operaciones con variables de entradas de punto flotante modifica el formato de las variables y el tipo de conversión [12].

2) *Instrucciones.* La base del algoritmo utilizado para evaluar el tiempo de ejecución de las instrucciones en la plataforma Delfino LaunchPad se encuentra en el CÓDIGO 2. En particular, las funciones para la adquisición de señales mediante el ADC consideran una configuración de trigger por software y modo de interrupción tardío. La adquisición inicia al forzar el estado del registro para el inicio de conversión, SOC, y finaliza al limpiar el estado una interrupción indicadora de término de adquisición. El procedimiento indicado y una previa configuración del ADC asegura una correcta adquisición.

C. Consideraciones de configuración Delfino LaunchPad TMS320F28379D.

- Durante la creación o edición de un proyecto es posible configurar el formato de salida relacionado con la interfaz binaria de aplicación ABI, ya sea Legacy (COFF) o EABI (ELF). La ABI define la interfaz de bajo nivel entre programas, sus componentes y entorno de ejecución, el fabricante indica que “el propósito es permitir a los proveedores de herramientas, software y usuarios de la MCU C2000 crear herramientas y programas que interoperen entre sí”. EABI es la interfaz

CÓDIGO 1

Código Delfino LaunchPad: Operaciones matemáticas, formato numérico Int

```
1 int a,b;
2 int c0,c1,c2,c3,c4;
3 int a0=5, b0=2;
4 float cFloat;
5 static int sc1,sc2;
6 static int sc3,sc5;
7
8 /*Declaración pines.
9 Salidas digitales
10 D0 a D6*/
11
12 int main(void)
13 {
14     while(1)
15     {
16         a = sc1+sc2;
17         b = sc3+sc4;
18
19         GPIO_writePin(D0,1);
20         GPIO_writePin(D0,0);
21
22         GPIO_writePin(D1,1);
23         c0 = a0+b0;
24         GPIO_writePin(D1,0);
25
26         GPIO_writePin(D2,1);
27         c1 = a+b;
28         GPIO_writePin(D2,0);
29
30         GPIO_writePin(D3,1);
31         c2 = a-b;
32         GPIO_writePin(D3,0);
33
34         GPIO_writePin(D4,1);
35         c3 = a*b;
36         GPIO_writePin(D4,0);
37
38         GPIO_writePin(D5,1);
39         c4 = a/b;
40         GPIO_writePin(D5,0);
41
42         GPIO_writePin(D6,1);
43         cFloat = (float)a;
44         GPIO_writePin(D6,0);
45
46         sc1=c1;
47         sc2=c2;
48         sc3=c3;
49         sc4=c4;
50     }
51 }
```

CÓDIGO 2

Código Delfino LaunchPad: Instrucciones

```
1 float VFloat, mFloat=5.1, cFloat=12.75;
2 int VInt, aInt=12;
3
4 /*Declaración de pines. Salidas digitales
5 D0 a D5*/
6
7 int main(void)
8 {
9     while(1)
10     {
11         GPIO_writePin(D0, 1);
12         GPIO_writePin(D0, 0);
13
14         GPIO_writePin(D1, 1);
15         ADC_forceSOC(ADCA_BASE,ADC_SOC_NUMBER0);
16         while(ADC_getInterruptStatus(ADCA_BASE,
17             ADC_INT_NUMBER1)==false){ }
18         ADC_clearInterruptStatus(ADCA_BASE,
19             ADC_INT_NUMBER1);
20         adcAResult0=ADC_readResult(ADCARESULT_BASE,
21             ADC_SOC_NUMBER0);
22         GPIO_writePin(D1, 0);
23
24         GPIO_writePin(D2, 1);
25         VFloat = mFloat*((float)adcAResult0)+cFloat;
26         GPIO_writePin(D2, 0);
27
28         GPIO_writePin(D3, 1);
29         VInt = (int)VFloat + aInt;
30         GPIO_writePin(D3, 0);
31
32         GPIO_writePin(D4, 0);
33         GPIO_writePin(D4, 1);
34
35         GPIO_writePin(D5, 1);
36         DAC_setShadowValue(DACB_BASE,adcAResult0);
37         GPIO_writePin(D5, 0);
38     }
39 }
```

actualizada y permite mayor compatibilidad con funciones, como archivos de objetos compartidos [27].

- La configuración del lugar de almacenamiento y unidad de procesamiento donde se ejecutará el algoritmo requiere identificar la CPU a utilizar, CPU1 o CPU2, y la memoria para el almacenamiento del programa principal, FLASH o RAM, para lo cual se debe crear el símbolo predefinido “_FLASH” o no indicar en caso de utilizar RAM. Lo anterior debe ser ingresado en la sección de *CCS Properties->Build->C2000 Composer->Predefined Symbols->Pre-define Name*. Al seleccionar FLASH, esta se utiliza principalmente como memoria para el programa principal y memoria de datos estática.
- La metodología puede ser implementada mediante la edición de un proyecto disponible dentro de la librería Drivelib\F28379D. En particular, es posible utilizar el proyecto “empty_projects” donde las operaciones e instrucciones propuestas operarán de forma correcta, a una frecuencia de 100 MHz de no incorporar el símbolo predefinido “_LAUNCHXL_F28379D”. El cual habilita la información característica de la placa Delfino LaunchPad TMS320F28379D, como el reloj externo de 10MHz, actualizando el reloj de la CPU (SYSCLK) a 200 MHz.
- La adquisición y conversión de señales mediante el ADC requiere seleccionar la resolución y una de las múltiples fuentes de trigger, en particular, se utiliza una resolución de 12 bits y una fuente basada en software (S/W, software immediate start), [28]. Adicionalmente, se debe seleccionar la frecuencia del ADCCLK mediante el pre-escalado de SYSCLK, `ADC_setupSOC()`.

IV. ADQUISICIÓN Y RESULTADOS

Los resultados obtenidos mediante la implementación de los algoritmos para la evaluación del tiempo de ejecución implementados en la plataforma de desarrollo Arduino Due y Delfino LaunchPad consideran las siguientes configuraciones iniciales.

- Arduino Due, DAC con resolución de 12 bits. Frecuencia de CPU de 84 MHz.
- Delfino LaunchPad, considera una frecuencia de CPU (SYSCLK) de 100 MHz, un reloj de 25 MHz en ADC (ADCCLK), resolución de 12 bits, trigger por software con interrupción tardía y una ventana de muestra de 15. Lo anterior utilizando la CPU1.

En caso de la plataforma Delfino LaunchPad se evalúan 4 casos distintos de configuración considerando la interfaz binaria utilizada (COFF o EABI) y el lugar de memoria donde se almacena el código principal (RAM o FLASH). Los 4 casos corresponden a (i) EABI-FLASH, (ii) COFF-FLASH, (iii) EABI-RAM y (iv) COFF-RAM. Para Arduino Due se considera su configuración por defecto, código en memoria FLASH.

A. Plataforma de adquisición

La adquisición de señales y su procesamiento para medir el tiempo de ejecución se implementa a través del instrumento Digital Discovery de Digilent, equipo que opera como analizador lógico o generador de patrones. Cuenta con 24 canales digitales como analizador lógico, de los cuales 8 se encuentran disponibles si se utiliza una frecuencia de trabajo de 800 MHz al conectar el adaptador de alta velocidad. Opción utilizada para la adquisición, obteniendo una resolución de 0.1 ns al evaluar las señales en el software WaveForms.

B. Resultados

Las mediciones realizadas son presentadas luego de procesar los datos según la metodología. En Tabla 4 se dan a conocer los tiempos de procesamiento de la operación suma de variables constantes, $a0+b0$, y actualizada en cada iteración, $a+b$, seguidas por las operaciones matemáticas en Tabla 5 e instrucciones en Tabla 6. Lo anterior para el caso de Arduino Due y las 4 configuraciones de memoria e interfaz binaria indicadas para Delfino LaunchPad, (i) EABI-FLASH, (ii) COFF-FLASH, (iii) EABI-RAM y (iv) COFF-RAM. Los resultados obtenidos desde Arduino Due corroboran el rendimiento del dispositivo al realizar operaciones matemáticas básicas, [12][18].

1) Operación suma de variables constantes y actualizadas en cada iteración, TABLA 4:

Los resultados en la plataforma Arduino Due dan a conocer un mayor tiempo de ejecución en las operaciones de punto flotante, respecto a la suma de variables enteras. El incremento corresponde a 4.6 veces el tiempo de una suma de variables enteras constantes y 17 veces si la operación se realiza con variables actualizadas en cada iteración.

Las operaciones con formato entero y flotante en la plataforma Delfino LaunchPad se ejecutan en un tiempo menor a 0.090 μ s, variando según el tipo de interfaz binaria y almacenamiento. En particular, la operación suma de variables actualizadas se ejecutó en 0.080 μ s y las de punto flotante en 0.010 μ s al utilizar almacenamiento en memoria FLASH y una interfaz EABI.

TABLA 4. TIEMPO OPERACIÓN MATEMÁTICA I

Formato numérico	int a, b, c const int a0=5, b0=2		float a, b, c const float a0=5.1, b0=2.1	
Operación	$c=a0+b0$	$c=a+b$	$c=a0+b0$	$c=a+b$
Arduino Due, μ s	0,273	0,071	1,285-1,286	1,274-1,393
Delfino TMS320F28379D, EABI, FLASH, μ s	0,040	0,080	0,090	0,010
Delfino TMS320F28379D, COFF, FLASH, μ s	0,040	0,050	0,040	0,040
Delfino TMS320F28379D, EABI, RAM, μ s	0,070	0,070	0,050	0,050
Delfino TMS320F28379D, COFF, RAM, μ s	0,060	0,050	0,050	0,060

TABLA 5. TIEMPO OPERACIÓN MATEMÁTICA II

Formato numérico	int a, b, c					float a, b, c				
	$c=a+b$	$c=a-b$	$c=a*b$	$c=a/b$	$float(a)$	$c=a+b$	$c=a-b$	$c=a*b$	$c=a/b$	$int(a)$
Arduino Due, μs	0,071	0,118	0,106	0,142-0,178	0,845	1,274-1,393	1,263-1,512	0,965-2,188	2,821-2,822	0,690-0,691
Delfino, EABI, FLASH , μs	0,080	0,040	0,080	0,560	0,040	0,010	0,040	0,040	0,570-0,600	0,040
Delfino, COFF, FLASH , μs	0,050	0,080	0,040	0,460	0,040	0,040	0,010	0,040	0,580-0,640	0,040
Delfino, EABI, RAM , μs	0,070	0,060	0,070	0,340	0,050	0,050	0,040	0,050	0,470-0,490	0,050
Delfino, COFF, RAM , μs	0,050	0,060	0,050	0,360	0,040	0,060	0,050	0,060	0,330	0,040

TABLA 6. TIEMPO INSTRUCCIONES

Instrucción	Tiempo ejecución instrucción, μs				
	Arduino Due	Delfino LaunchPad			
		FLASH		RAM	
		EABI	COFF	EABI	COFF
Vin = analogRead(pin, Value)	4,405- 5,582	2,360	2,360	3,230	1,690
Vfloat= mFloat *float(Vin) +bFloat	2,571* - 3,750	0,080	0,080	0,100	0,080
Vint = int(Vfloat) + bInt	0,667 -1,798	0,080	0,080	0,100	0,070
digitalWrite(pin,1), on	2,164 - 2,166	0,650	0,650	0,820	0,460
digitalWrite(pin,0), off	2,105 - 2,155	0,510	0,510	0,750	0,390
analogWrite(DAC, Vint)	3,536* - 4,706	0,350	0,350	0,790	0,270

2) Operaciones matemáticas enteras y punto flotante, Tabla 5:

Para Arduino Due las operaciones resta y multiplicación de variables enteras se ejecutan en un tiempo de $\sim 0.118 \mu s$, tiempo que se incrementa en ~ 11 veces al operar con variables de punto flotante. En cuanto a la división, la ejecución con variables de punto flotante demora 17 veces más respecto a la entera ($2.8 \mu s$ respecto a $0.178 \mu s$).

Launchad Delfino presenta tiempos de ejecución iguales o menores a $0.08 \mu s$ en operaciones de suma, resta, multiplicación y conversión de formato, tanto para variables enteras como de punto flotante. De igual forma la división mantiene un tiempo de procesamiento de $\sim 0.57 \mu s$ al utilizar EABI y FLASH, en cambio al utilizar almacenamiento en RAM las variaciones de los tiempos son mayores según el formato numérico. La conversión de formato int a float y viceversa es fija sin importar el tipo de conversión ($\sim 0.04 \mu s$).

3) Instrucciones, Tabla 6:

En plataforma Arduino Due la instrucción de lectura de una señal analógica alcanza un tiempo de $\sim 5.58 \mu s$, $2.1 \mu s$ para modificar el estado de una salida digital y $4.7 \mu s$ al cambiar la salida analógica desde el DAC.

Para la plataforma Delfino LaunchPad los tiempos de ejecución de una adquisición de señal analógica, modificación de salida digital y salida analógica corresponden a $2.36 \mu s$, $0.65 \mu s$ y $0.35 \mu s$ respectivamente, considerando almacenamiento en FLASH e interfaz EABI. El tiempo de ejecución de las instrucciones en Delfino LaunchPad almacenadas en FLASH no difieren de la interfaz binaria. Sin embargo, los tiempos incrementan al almacenar en RAM en caso de EABI. Por el contrario, los tiempos disminuyen al utilizar la interfaz COFF y almacenamiento en RAM.

V. COMPARACIÓN DE DESEMPEÑO

Las operaciones matemáticas básicas de punto flotante como la suma, la resta y la multiplicación se ejecutan a lo menos 16 veces más rápido en Delfino LaunchPad, disminuyendo los

tiempos de ejecución desde el rango $1.0-2.8 \mu s$ al rango $0.01-0.06 \mu s$, Tabla 4 y Tabla 5. La operación división de punto flotante se ejecuta ~ 5 veces más rápido, disminuyendo los tiempos de $2.82 \mu s$ a $0.57 \mu s$. Las mejoras en los tiempos de ejecución de funciones con variables en formato numérico flotante en Delfino LaunchPad son efecto de utilizar la FPU alojada en su microcontrolador, en comparación a la ejecución por software realizada por Arduino Due.

Por otro lado, al utilizar Delfino LaunchPad las operaciones con variables enteras, resta y multiplicación, se ejecutan a lo menos 1.5 veces más rápido que con Arduino Due. En el caso de la suma y división el tiempo de ejecución es similar o aumenta respecto a Arduino Due. En particular, en Delfino LaunchPad es posible considerar que los tiempos de ejecución de divisiones enteras y punto flotante son inferiores a $0.6 \mu s$ al utilizar EABI-FLASH, existiendo la posibilidad de mejorar sus valores al utilizar módulos especializados (vía hardware).

Respecto a la conversión de formato de variables enteras a punto flotante y viceversa, en Delfino LaunchPad estas se ejecutan 21 y 17 veces más rápido que en Arduino Due, respectivamente. Manteniendo un tiempo de ejecución fijo de $0.04 \mu s$ para ambas operaciones, Tabla 5. En Arduino Due los tiempos varían según el tipo de conversión, $0.845 \mu s$ y $0.69 \mu s$ respectivamente.

Las instrucciones evaluadas en la Tabla 6 permiten indicar que el tiempo de adquisición de una señal analógica, por medio de la activación de un DAC por software, junto al de escalamiento por medio de $Vfloat=mFloat*float(Vin)+bFloat$, varía entre $\sim 7 \mu s$ y $\sim 9 \mu s$ en el caso de Arduino Due, y es igual a $2.44 \mu s$ al utilizar la tarjeta Delfino LaunchPad (almacenamiento en FLASH). Lo que implica ejecutar la adquisición y cálculo entre 2.9 y 3.7 veces más rápido. Sin embargo, si se requiere transformar la variable indicada a un formato entero a través de $Vint=int(Vfloat)+bInt$, Arduino Due demorará en total un tiempo entre $\sim 7.6 \mu s$ y $\sim 11 \mu s$, y Delfino LaunchPad $2.52 \mu s$. Obteniendo el resultado de 3 y 4 veces más rápido con Delfino LaunchPad.

En cuanto a las salidas análogas por DAC, Delfino LaunchPad puede ejecutar la instrucción en un tiempo de 0.35 μ s considerando almacenamiento en memoria FLASH. Por lo que es 10 veces más rápido que Arduino Due. En el caso de salidas digitales, la velocidad de cambio de estado se incrementa a lo menos en 3 veces, ejecutándose en Delfino LaunchPad en 0,65 μ s.

Al evaluar el desempeño de las tarjetas de desarrollo respecto a las instrucciones indicadas en Tabla 6, en Delfino LaunchPad es necesario tener presente la interfaz binaria configurada en el proyecto de CCS y el lugar de almacenamiento del programa principal. EABI es la interfaz recomendada por el fabricante, ya que permite utilizar las funciones o módulos de cálculo avanzados.

Finalmente, se debe tener presente que es posible mejorar los tiempos de ejecución en la plataforma Delfino LaunchPad mediante el aumento de la frecuencia de operación de la CPU desde 100 a 200 MHz, SYSCLK, y parámetros del ADC como la frecuencia y la ventana de muestreo. Una opción más avanzada y complementaria a la anterior es utilizar los módulos dedicados disponibles en el microcontrolador, unidades como la CLA, VCU y TMU en caso de requerirse.

VI. CONCLUSIÓN

Este trabajo presenta un análisis y comparación del desempeño de las tarjetas de desarrollo Arduino Due y Delfino LaunchPad F28379D, considerando la compatibilidad eléctrica y posibilidad de incrementar la capacidad de cómputo al utilizar la placa Delfino LaunchPad. Se detallan los periféricos disponibles, complejidades de configuración en los IDE y se analizan sus capacidades de procesamiento. La información sobre la disponibilidad de periféricos en los terminales de las tarjetas de desarrollo demuestra la compatibilidad, sin embargo, existe un incremento en la complejidad de configuración inicial de la plataforma Delfino LaunchPad.

La migración desde Arduino Due a Delfino LaunchPad permite disminuir los tiempos de ejecución de operaciones básicas de punto flotante como la suma, la resta y la multiplicación a lo menos en 16 veces considerando una frecuencia de operación de 100MHz en Delfino LaunchPad, debido al uso de la FPU disponible en su microcontrolador. Manteniendo la compatibilidad eléctrica entre la aplicación y la tarjeta. Esto implica considerar tiempos de ejecución en el rango 0.01- 0.06 μ s. En caso de utilizar variables enteras la operación resta y multiplicación se ejecutan 1.5 veces más rápido al utilizar Delfino LaunchPad. En el caso de la suma y división el tiempo de ejecución es similar o aumenta respecto a Arduino Due, Tabla 5.

La migración de Arduino Due a Delfino LaunchPad incrementa la velocidad en que se ejecuta la adquisición y procesamiento de una señal análoga para luego utilizarla en el proyecto. Se puede considerar que el aumento es de hasta 3,7 veces si se realiza un escalamiento considerando una variable de punto flotante. Entre $\sim 7 \mu$ s y $\sim 9 \mu$ s para Arduino Due y 2.44 μ s para Delfino LaunchPad. La configuración del ADC considera trigger por software, modo de interrupción tardío, ventana de muestreo igual a 15 y ADCCLK 25 MHz.

Finalmente, para las salidas análogas por DAC, Delfino LaunchPad puede ejecutar la instrucción de escritura 10 veces más rápido que Arduino Due, en 0.35 μ s. En el caso de salidas digitales, la velocidad de cambio de estado se incrementa en ~ 3 veces, ejecutándose en Delfino LaunchPad en $\sim 0,65 \mu$ s.

AGRADECIMIENTOS

Los autores desean agradecer el apoyo financiero de la Agencia de Investigación y Desarrollo Chilena, ANID, a través del proyecto ANID/FONDECYT/1201683, y la MacroFacultad de Ingeniería 2030, Chile.

REFERENCIAS

- [1] M. A. Jamshed, K. Ali, Q. H. Abbasi, M. A. Imran, and M. Ur-Rehman, "Challenges, Applications, and Future of Wireless Sensors in Internet of Things: A Review," *IEEE Sens. J.*, vol. 22, no. 6, pp. 5482–5494, Mar. 2022, doi: 10.1109/JSEN.2022.3148128.
- [2] L. Mocerino and A. Calimera, "Fast and Accurate Inference on Microcontrollers with Boosted Cooperative Convolutional Neural Networks (BC-Net)," *IEEE Trans. Circuits Syst. I Regul. Pap.*, vol. 68, no. 1, pp. 77–88, Jan. 2021, doi: 10.1109/TCSI.2020.3039116.
- [3] X. Xiang, X. Zhang, and H. Chen, "Acquisition and Enhancement of Remote Human Vocal Signals Based on Doppler Radar," *IEEE Sens. J.*, vol. 21, no. 18, pp. 20348–20361, Sep. 2021, doi: 10.1109/JSEN.2021.3094630.
- [4] J. A. Antonino-Daviu, K. N. Gyftakis, R. Garcia-Hernandez, H. Razik, and A. J. M. Cardoso, "Comparative influence of adjacent and non-adjacent broken rotor bars on the induction motor diagnosis through MCSA and ZSC methods," *IECON 2015 - 41st Annu. Conf. IEEE Ind. Electron. Soc.*, pp. 1680–1685, 2015, doi: 10.1109/IECON.2015.7392343.
- [5] S. He, D. Zhou, X. Wang, Z. Zhao, and F. Blaabjerg, "A Review of Multisampling Techniques in Power Electronics Applications," *IEEE Trans. Power Electron.*, vol. 37, no. 9, pp. 10514–10533, Sep. 2022, doi: 10.1109/TPEL.2022.3169662.
- [6] E. J. Pino, D. E. Arias, P. Aqueveque, P. Melin, and D. W. Curtis, "Wireless pilot monitoring system for extreme race conditions," *Proc. Annu. Int. Conf. IEEE Eng. Med. Biol. Soc. EMBS*, pp. 5014–5017, 2012, doi: 10.1109/EMBC.2012.6347119.
- [7] D. E. Arias, E. J. Pino, P. Aqueveque, and D. W. Curtis, "Unobtrusive Support System for Prevention of Dangerous Health Conditions in Wheelchair Users," *Mob. Inf. Syst.*, vol. 2016, 2016, doi: 10.1155/2016/4568241.
- [8] D. Ibrahim and A. Davies, "The evolution of digital signal processors," *2019 6th IEEE Hist. Electrotechnol. Conf. HISTELCON 2019*, pp. 25–29, Sep. 2019, doi: 10.1109/HISTELCON47851.2019.9040130.
- [9] A. HajiRassouliha, A. J. Taberner, M. P. Nash, and P. M. F. Nielsen, "Suitability of recent hardware accelerators (DSPs, FPGAs, and GPUs) for computer vision and image processing algorithms," *Signal Process. Image Commun.*, vol. 68, pp. 101–119, Oct. 2018, doi: 10.1016/J.IMAGE.2018.07.007.
- [10] S. Banerjee and A. Sinha, "Performance analysis of different DSP algorithms on advanced microcontroller and FPGA," *2009 Int. Conf. Adv. Comput. Tools Eng. Appl. ACTEA 2009*, pp. 609–613, 2009, doi: 10.1109/ACTEA.2009.5227848.
- [11] F. Ortega-Zamorano, J. M. Jerez, D. U. Munoz, R. M. Luque-Baena, and L. Franco, "Efficient Implementation of the Backpropagation Algorithm in FPGAs and Microcontrollers," *IEEE Trans. Neural Networks Learn. Syst.*, vol. 27, no. 9, pp. 1840–1850, Sep. 2016, doi: 10.1109/TNNLS.2015.2460991.
- [12] O. E. Amestica, P. E. Melin, C. R. Duran-Faundez, and G. R. Lagos, "An Experimental Comparison of Arduino IDE Compatible Platforms for Digital Control and Data Acquisition Applications," *IEEE Chil. Conf. Electr. Electron. Eng. Inf. Commun. Technol. CHILECON 2019*, Nov. 2019, doi: 10.1109/CHILECON47746.2019.8986865.
- [13] V. V. Rankovska, "Review of Open-source Microcontroller and Programmable Logic Development Boards," Sep. 2021, DOI:

10.1109/ET52713.2021.9580150.

- [14] F. J. Diaz, M. A. Osorio, A. P. Amadeo, and F. E. Lopez, "Internet of Things as a mean to motivate freshmen students in the 'Facultad de Informática' of the National University of la Plata (UNLP)," *Proc. - Int. Conf. Chil. Comput. Sci. Soc. SCCC*, vol. 2019-Novem, Nov. 2019, doi: 10.1109/SCCC49216.2019.8966415.
- [15] M. Lineros *et al.*, "Electronics for everybody: Student practical experiences using Arduino," in *Proceedings - International Conference of the Chilean Computer Science Society. SCCC*, Jul. 2018, vol. 2018-Novem, doi: 10.1109/SCCC.2018.8705228.
- [16] C. Vidal-Silva, J. Serrano-Malebran, and F. Pereira, "Scratch and Arduino for Effectively Developing Programming and Computing-Electronic Competences in Primary School Children," *Proc. - Int. Conf. Chil. Comput. Sci. Soc. SCCC*, vol. 2019-November, Nov. 2019, doi: 10.1109/SCCC49216.2019.8966401.
- [17] M. J. Ibarra, E. Alcarraz, O. Tapia, Y. P. Atencio, Y. Mamani-Coaquira, and H. A. Huillcen Baca, "NFT-I technique using IoT to improve hydroponic cultivation of lettuce," *Proc. - Int. Conf. Chil. Comput. Sci. Soc. SCCC*, vol. 2020-November, Nov. 2020, doi: 10.1109/SCCC51225.2020.9281277.
- [18] P. Melin, C. Baier, E. Espinosa, J. Riedemann, J. Espinoza, and R. Pena, "Study of the Open-Source Arduino DUE Board as Digital Control Platform for Three-Phase Power Converters," *IEEE Access*, vol. 10, pp. 7574–7587, 2022, doi: 10.1109/ACCESS.2021.3138705.
- [19] D. Apablaza and J. Munoz, "Laboratory Implementation of a Boost Interleaved Converter for PV Applications," *IEEE Lat. Am. Trans.*, vol. 14, no. 6, pp. 2738–2743, Jun. 2016, doi: 10.1109/TLA.2016.7555247.
- [20] J. Riedemann *et al.*, "Design and building of an automatic alternator synchronizer based on open-hardware arduino platform," *IEEE Access*, vol. 7, pp. 105116–105122, 2019, doi: 10.1109/ACCESS.2019.2932294.
- [21] X. Quan and A. Q. Huang, "PI-Based Synchronous Reference Frame Frequency-Locked Loop," *IEEE Trans. Ind. Electron.*, vol. 68, no. 5, pp. 4547–4553, May 2021, doi: 10.1109/TIE.2020.2985002.
- [22] Z. Farooq, T. Zaman, M. A. Khan, Nasimullah, S. M. Muyeen, and A. Ibeas, "Artificial neural network based adaptive control of single phase dual active bridge with finite time disturbance compensation," *IEEE Access*, vol. 7, pp. 112229–112239, 2019, doi: 10.1109/ACCESS.2019.2934253.
- [23] L. N. Nguyen and T. C. Pham, "Optimal Tracking Control for PMSM with Partially Unknown Dynamics, Saturation Voltages, Torque, and Voltage Disturbances," *IEEE Trans. Ind. Electron.*, vol. 69, no. 4, pp. 3481–3491, Apr. 2022, doi: 10.1109/TIE.2021.3075892.
- [24] A. Bakeer, I. S. Mohamed, P. B. Malidarreh, I. Hattabi, and L. Liu, "An Artificial Neural Network-Based Model Predictive Control for Three-Phase Flying Capacitor Multilevel Inverter," *IEEE Access*, vol. 10, pp. 70305–70316, 2022, doi: 10.1109/ACCESS.2022.3187996.
- [25] G. Boztas and O. Aydogmus, "Implementation of sensorless speed control of synchronous reluctance motor using extended Kalman filter," *Eng. Sci. Technol. an Int. J.*, vol. 31, p. 101066, Jul. 2022, doi: 10.1016/J.JESTCH.2021.09.012.
- [26] N. K. Buduru, S. B. Karanki, and S. Manikandan, "DSP implementation of hilbert transform for voltage variation tracking," *2020 7th Int. Conf. Signal Process. Integr. Networks, SPIN 2020*, pp. 1039–1043, Feb. 2020, doi: 10.1109/SPIN48934.2020.9071150.
- [27] "C2000 Migration from COFF to EABI." https://software-dl.ti.com/ccs/esd/documents/C2000_c28x_migration_from_coff_to_eabi.html (accessed Sep. 09, 2022).
- [28] Texas Instruments, *TMS320F2837xD Dual-Core Microcontrollers: Technical Reference Manual, Literature Number: SPRUHM81. Revised September 2019.* 2013.